Name: Mohaiya aldossary
Student ID: 2230004579

Lab 7 — Large Language Models (LLMs)

The Three Main Transformer Model Families

Family	Example Models	Main Use
Encoder-only	BERT, RoBERTa	Classification, sentiment analysis, NER
Encoder–decoder	T5, BART	Translation, summarization, question answering
Decoder-only	GPT family	Text generation, conversation, completion

1. Setup

In [1]:
```
%pip -q install transformers sentencepiece torch
```

```
Note: you may need to restart the kernel to use updated packages.
```

In [2]:
```
import os
os.environ["USE_TF"] = "0"

from transformers import pipeline, set_seed

print("Libraries loaded successfully!")
```

```
Libraries loaded successfully!
```

Part A — Decoder-Only Model

Decoder-only example: GPT-style Text Generation

A decoder-only model predicts the next token based on previous tokens.

We will use **DistilGPT-2**, a small GPT-style model.

In [3]:
```
generator = pipeline("text-generation", model="distilgpt2")

prompt = "Natural language processing is"

output = generator(prompt, max_new_tokens=30, do_sample=False)

print(output[0]["generated_text"])
```

```
Natural language processing is a common feature of the language processing process.
```

Part B — Encoder–Decoder Model

Encoder–decoder example: Instruction / Text-to-Text Generation

Encoder–decoder models contain two main parts:

1. The **encoder** reads the input.
2. The **decoder** produces the output.

We will use **FLAN-T5-small**.

In [4]:

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

tokenizer = AutoTokenizer.from_pretrained("google/flan-t5-small")
model = AutoModelForSeq2SeqLM.from_pretrained("google/flan-t5-small")

prompt = "Translate to French: I love natural language processing."

# Convert text into tokens
output = model.generate(**tokenizer(prompt, return_tensors="pt"))

# Convert tokens back to text
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

```
Je s'agissant de la technologie de linguage naturelle.
```

Part C — Encoder-Only Model

Encoder-only example: Sentiment Classification

Encoder-only models read the full input and create a representation of it.

They are very useful when the task is to **understand or classify text**.

Here we use a DistilBERT sentiment model.

In [5]:

```
classifier = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english")

text = "This NLP lab is easy and interesting."

result = classifier(text)
print(result)
```

```
[{'label': 'POSITIVE', 'score': 0.999778687953949}]
```

Tasks

Complete the following tasks.

Task 1 — Model Families

For each task below, choose the most suitable model family:

1. Sentiment classification
2. Machine translation
3. Open-ended text generation
4. Named Entity Recognition
5. Text summarization

In [6]:

```
families = {
    "Sentiment classification": "Encoder-only",
    "Machine translation": "Encoder-decoder",
    "Open-ended text generation": "Decoder-only",
    "Named Entity Recognition": "Encoder-only",
    "Text summarization": "Encoder-decoder",
}

for task, family in families.items():
    print(f"{task}: {family}")
```

```
Sentiment classification: Encoder-only
Machine translation: Encoder-decoder
Open-ended text generation: Decoder-only
Named Entity Recognition: Encoder-only
Text summarization: Encoder-decoder
```

Task 2 — Encoder-only Model

Use the sentiment classifier on these three sentences:

```
The movie was excellent.
The lecture was difficult to understand.
I enjoyed learning about transformers.
```

Print the predicted label and score for each sentence.

In [7]:

```
sentences = [
    "The movie was excellent.",
    "The lecture was difficult to understand.",
    "I enjoyed learning about transformers.",
]

for sentence, prediction in zip(sentences, classifier(sentences)):
    print(sentence)
    print(f"Label: {prediction['label']}, Score: {prediction['score']:.4f}")
```

```
The movie was excellent.
Label: POSITIVE, Score: 0.9999
The lecture was difficult to understand.
Label: NEGATIVE, Score: 0.9997
I enjoyed learning about transformers.
Label: POSITIVE, Score: 0.9997
```

Task 3 — Encoder–Decoder Model

Use the FLAN-T5 model to perform the following:

- Summarize a short paragraph.

In [8]:

```
paragraph = (
    "A university library introduced a digital borrowing system. "
    "Students can now reserve books online and receive reminders before "
    "their return dates. The new system has reduced waiting times and "
    "made it easier for students to find the books they need."
)

inputs = tokenizer("Summarize: " + paragraph, return_tensors="pt")
summary = model.generate(**inputs, max_new_tokens=60, do_sample=False)
print("Paragraph:", paragraph)
print("Summary:", tokenizer.decode(summary[0], skip_special_tokens=True))
```

```
Paragraph:
 A university library introduced a digital borrowing system. Students can now reserve books online and receive reminders before their return dates. The new system has reduced waiting times and made it easier for students to find the books they need.
Summary: A new system has been introduced to help students reserve books online.
```

Task 5 — Short Reflection

Answer in 3–5 sentences:

Why is a decoder-only model suitable for a chatbot such as ChatGPT?

A decoder-only model generates a response one token at a time using the conversation so far as context. Its causal attention lets each token depend on earlier tokens, making it suitable for continuing a dialogue. Training on large amounts of text helps it learn language patterns and produce coherent responses. Instruction tuning and feedback further help it follow user requests and respond appropriately.

Optional Challenge

Try generating text several times with the same prompt.

Remove:

```
set_seed(42)
```

and run the sampling cell multiple times.

Question

Why do you sometimes get different outputs even when the prompt is the same?

Hint: Sampling introduces randomness when choosing the next token.

In [9]:

```
prompt = "Learning about transformers is"

for run in range(3):
    generated = generator(prompt, max_new_tokens=30, do_sample=True,
                          pad_token_id=generator.tokenizer.eos_token_id)
    print(f"Run {run + 1}: {generated[0]['generated_text']}")
```

```
Run 1: Learning about transformers is a good starting point for exploring the ways that the Internet can be used to improve the lives of the human being: the social networks that exist to provide
Run 2: Learning about transformers is an amazing discovery.

Run 3: Learning about transformers is a useful and useful tool for building a functional, reusable system.
```

Sampling randomly selects each next token from the model's probability distribution. Without resetting a fixed seed, the random choices can differ between runs, producing different continuations of the same prompt.